PROJECT REPORT

STUDENT PASS/FAIL PREDICTION SYSTEM USING MACHINE LEARNING

EduPredict

Submitted in partial fulfilment of the requirements for
Master of Computer Applications (MCA)
Semester III · Course Code MCA-521 – Research Project
Kavayitri Bahinabai Chaudhari North Maharashtra University, Jalgaon
Faculty of Science and Technology · W.E.F. 2024–25

Project Guide: _____
Student Name(s): _____
Academic Year: _____

Abstract

PROJECT REPORT

Keywords: educational data mining, student performance, classification, Flask, scikit-learn, early warning, decision support.

Table of Contents

Chapter 1: Introduction

Project Introduction

Background of the Project

Problem Statement

Objectives

Scope of the Project

Chapter 2: Literature Review

Review of Previous Research

Study of Existing Systems

Chapter 3: System Analysis and Methodology

Proposed System

Research Methodology

Flowchart

System Architecture

Chapter 4: System Design

Database Design

UML Diagrams

System Modules

System Design Diagrams

Chapter 5: Software and Hardware Requirements

Software Requirements

Hardware Requirements

Programming Languages

Tools and Technologies Used

Chapter 6: Implementation and Testing

Project Implementation

Module Description

Testing Methods

Chapter 7: Results and Conclusion

Output Screenshots

Results and Observations

Conclusion

Chapter 8: Future Scope and References

Future Enhancements

References

Chapter 1: Introduction

Project Introduction

EduPredict is a web-based academic decision-support application that estimates a student's likely examination outcome from academic and engagement indicators. The project combines a trained binary classifier with a student information system, administrative dashboards, prediction history, early-warning views, and a student self-service portal. The implementation is intended as a working MCA research project rather than a static model demonstration: inputs can be entered through the browser, inference is performed by the saved model, and the result is presented with probabilities and contextual guidance.

Background of the Project

Higher-education institutions collect marks, attendance, assignment completion and related information throughout a semester. These signals are often reviewed in separate spreadsheets and usually receive concentrated attention near examinations. Educational data mining and learning analytics investigate how such records can reveal patterns relevant to achievement and timely support. The present project applies supervised classification to historical labelled records and exposes its output through a small, locally deployable application. Its purpose is early identification and human follow-up, not automated grading.

Problem Statement

When student indicators are distributed across manual records, staff may not notice a combination of low assessment performance, poor attendance, incomplete assignments and backlogs until recovery opportunities have narrowed. A useful support system should combine these signals consistently, provide a repeatable estimate, preserve an auditable history and make the result understandable to administrators and students. The problem addressed is how to implement such a data-driven workflow while keeping the model output advisory and the application usable in a resource-limited institutional setting.

Objectives

The objectives are to: (1) prepare an 11-feature binary classification workflow; (2) compare four supervised classifiers using held-out evaluation and cross-validation; (3) select and persist a model and feature scaler; (4) provide validated student records and prediction logs; (5) show probability-based results, cohort summaries and intervention prompts; (6) provide separate administrator and student access; and (7) deliver a repeatable SQLite demonstration alongside MySQL configuration for deployment.

Scope of the Project

The implemented scope covers an academic-record dataset of 2,000 rows, an 80:20 stratified train/test split, Logistic Regression, RBF SVM, Gradient Boosting and Random Forest comparisons, Flask routes and templates, SQLAlchemy persistence, access controls,

a REST API, demo data seeding and downloadable report assets. The current demonstration uses synthetic/curated example student profiles and a previously trained model. It is not an admissions system, official examination authority, causal analysis, or a replacement for faculty assessment. Institutional policy, local validation and privacy review are required before operational use.

Chapter 2: Literature Review

Review of Previous Research

Educational data mining studies apply statistical learning to student records for tasks such as performance prediction, risk identification and resource planning. Romero and Ventura survey the field and describe the data-mining cycle from educational data preparation through model interpretation. Baker and Inventado distinguish educational data mining from broader learning analytics while noting their shared interest in extracting actionable insight from learning environments. In supervised prediction, the target definition, feature quality, representative sampling and validation design strongly affect whether results generalize. Classification metrics must therefore be interpreted together: accuracy alone can conceal errors on the less frequent class, while ROC-AUC measures ranking discrimination over thresholds.

Study of Existing Systems

Manual spreadsheet review is inexpensive and flexible, but relies on staff repeatedly joining records and applying informal thresholds. Learning-management dashboards summarize activity, yet may not combine examination outcomes, backlogs and attendance in a single risk estimate. General-purpose machine-learning notebooks can compare models but are less accessible to non-technical staff and do not inherently provide student accounts or persistent prediction history. EduPredict integrates these functions in a modest local web application. Unlike a production institutional platform, its dataset and demonstration accounts are illustrative; its outputs need local calibration and governance before decisions are made from them.

Chapter 3: System Analysis and Methodology

Proposed System

The proposed application accepts ten directly supplied indicators and derives average subject marks as the eleventh feature. A fitted StandardScaler transforms the feature vector, after which the selected classifier estimates the PASS class and probability. Flask services isolate inference and recommendations from route handling. Student and prediction records are stored through SQLAlchemy in SQLite Demo Mode or a configured MySQL database. The UI provides an interactive predictor, administrator functions, portal access, prediction history and model metrics.

Research Methodology

The modelling work uses 2,000 labelled records with 11 numerical predictors and a binary result (PASS 1: 1,343; FAIL 0: 657). Data are separated using stratified 80:20 sampling with random_state 42, producing 1,600 training and 400 test observations. Features are standardized. Four classifiers are compared, with the project metadata identifying Logistic Regression as the selection based on ROC-AUC. Five-fold cross-validation is reported on the training process. Metrics on the held-out test set include accuracy, precision, recall, F1 and ROC-AUC; a confusion matrix is retained. The selected model's stored evaluation is an estimate for this dataset and split, not evidence of external validity. The web layer validates ranges and displays probabilities, while a human reviewer is expected to determine any intervention.

Flowchart

The prediction flow is: select or enter a student; validate required fields and permitted ranges; construct the feature vector in the training order; apply the stored scaler; infer a class and probability distribution; persist a timestamped prediction snapshot; and show the result with guidance. Invalid or unavailable model states are reported instead of silently producing a result.

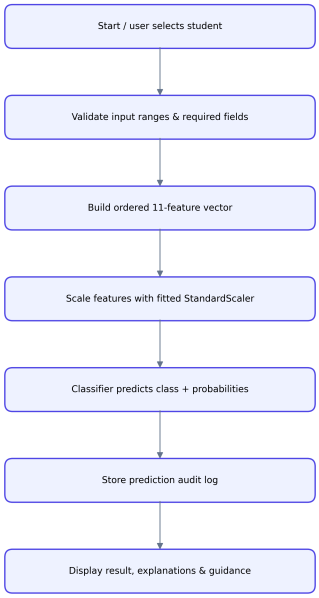


Figure 3.1: EduPredict prediction processing flow.

System Architecture

The application follows a Flask application-factory architecture with route blueprints, service-layer logic, SQLAlchemy models and Jinja templates. Flask-Login protects administrative views, Flask-WTF supplies CSRF protection, and ProxyFix supports deployment behind a trusted reverse proxy. The ML service lazily loads serialized estimator, scaler and metadata artifacts. The demo launcher selects SQLite and seeds sample data; a configured MySQL URI remains available for deployment. See Figure 3.2 for the logical component architecture.

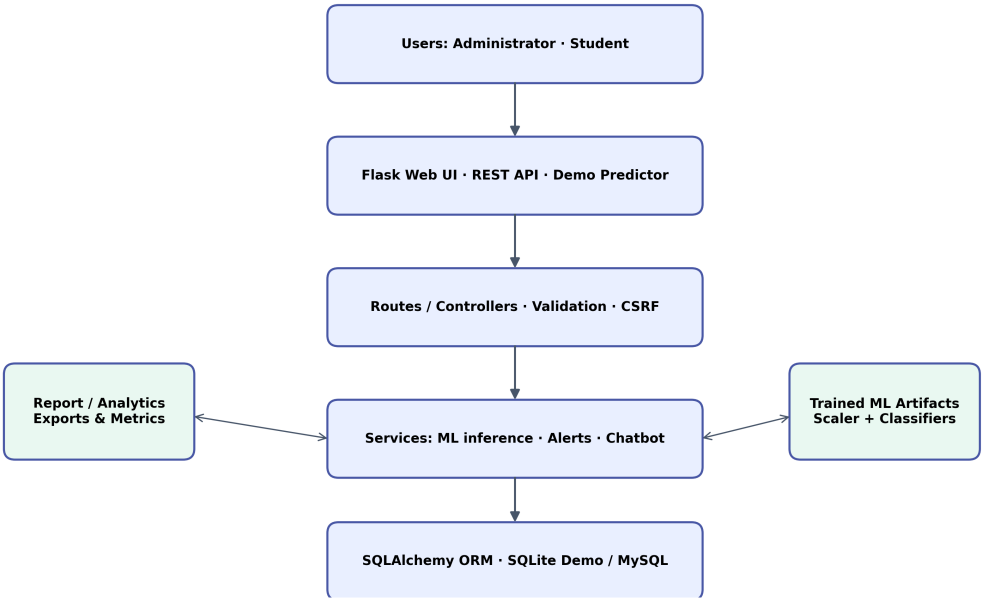


Figure 3.2: Logical system architecture and service boundaries.

Chapter 4: System Design

Database Design

The relational design contains four principal entities. Admin stores privileged account credentials. Student stores one profile per institution identifier. PredictionLog stores immutable prediction results, probabilities, model context, feature snapshot and time; one student may have many prediction logs. StudentAuth stores a separate student portal identity linked one-to-one with a student. Unique constraints protect usernames and student identifiers; foreign keys express record ownership. Passwords are stored as salted hashes through Werkzeug helpers rather than as plaintext. The ER-style data model is shown in Figure 4.1.

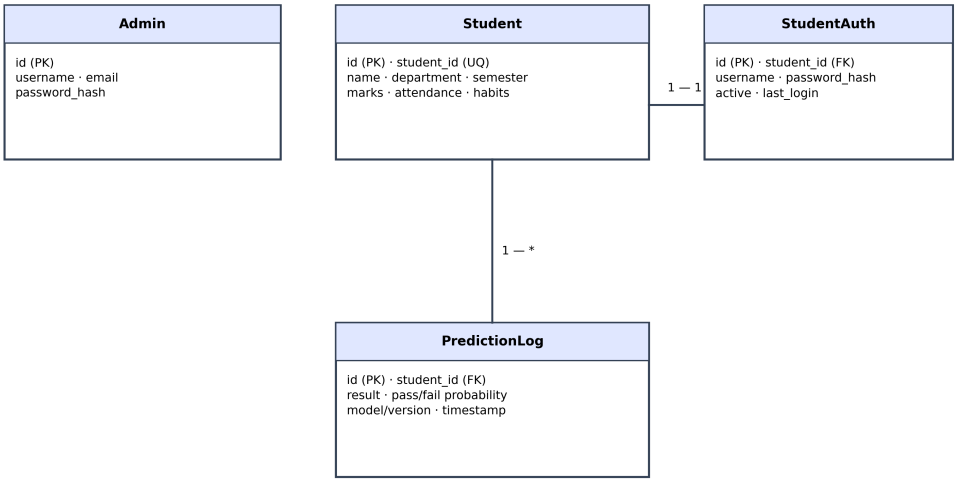


Figure 4.1: Principal entities and cardinality in the relational data model.

UML Diagrams

The conceptual use cases are: an administrator authenticates, manages student profiles, requests predictions, reviews cohort outcomes and prediction history, and examines alerts; a student authenticates to view only their own academic profile and result; a visitor uses the public demo predictor without changing stored student data. The component and data-model diagrams in this report document the principal actors, service boundaries and entity relationships. Authentication, validation, inference and persistence are separate responsibilities to reduce coupling.

System Modules

Authentication module: administrator login/logout and student portal sessions. Student management module: create, view, update and list academic profiles. Prediction module: validate feature inputs, call MLService and persist PredictionLog. Analytics module:

aggregate pass/fail counts, risk views and model metrics. Bulk module: import supported records with validation. Chatbot module: provide menu-based study guidance and live model statistics. Demo module: present interactive sliders/presets, expose real-time inference, provide quick demo access and reset the seeded SQLite dataset. Reporting module: generate the MCA-521 report and supporting evaluation figures.

System Design Diagrams

Figure 3.1 documents the ordered prediction workflow; Figure 3.2 presents the application architecture; Figure 4.1 presents the database entity relationships. Together they show the data path from browser input through validation and feature scaling to inference, persistence and display. The design deliberately keeps probability estimates distinct from institutional decisions.

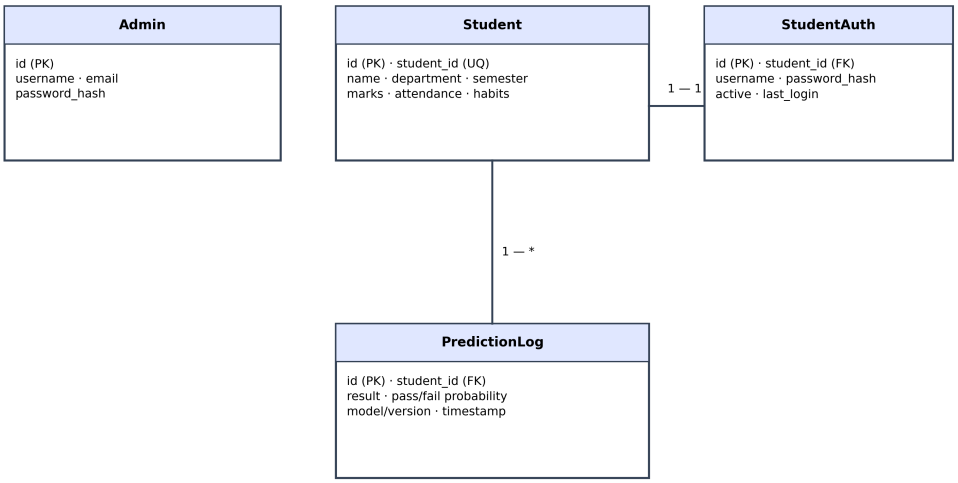


Figure 4.2: Data-model view supporting the design description.

Chapter 5: Software and Hardware Requirements

Software Requirements

The verified development environment is Debian GNU/Linux 12, Python 3.11 and Flask 3.0.3. The application uses Flask-SQLAlchemy, Flask-Login, Flask-WTF, Flask-CORS, SQLAlchemy, PyMySQL, scikit-learn, NumPy, pandas and joblib. SQLite is used by the self-contained demo; MySQL is supported through configuration. A modern browser with JavaScript enabled is required for the interactive predictor. A Python virtual environment is recommended for dependency isolation.

Hardware Requirements

For a classroom demonstration, a dual-core 64-bit processor, 4 GB RAM and approximately 1 GB free disk space are sufficient for the pre-trained artifacts and local SQLite database. Model retraining benefits from 8 GB RAM and additional free disk space. A network connection is needed only for package installation or remote access; the demo itself can run locally. These are practical project recommendations, not benchmarked minimum hardware specifications.

Programming Languages

Python implements server routes, validation, data access, machine-learning inference and report generation. HTML/Jinja structures server-rendered pages; CSS styles layouts; JavaScript handles interactive feature controls and asynchronous JSON requests. SQL is used indirectly through SQLAlchemy ORM and can be applied to MySQL deployment.

Tools and Technologies Used

Flask provides the web application framework; SQLAlchemy models relational data; scikit-learn trains and evaluates classifiers; pandas and NumPy support data preparation; joblib serializes trained artifacts; Bootstrap and Bootstrap Icons provide UI primitives; pytest validates behavior; python-docx and ReportLab create report deliverables; Matplotlib produces high-resolution evaluation and design figures. Git provides source version control.

Chapter 6: Implementation and Testing

Project Implementation

The repository is organized as a Flask application package, route blueprints, ORM models, services, templates, static assets, machine-learning scripts and saved artifacts. The demo launcher sets a demo environment, checks for trained artifacts, creates the application with DemoConfig and binds to all interfaces. Application initialization creates database tables, a default admin account and 30 sample student profiles when the demo database is empty. Each profile receives a prediction from the actual loaded MLService and an audit log; initial student accounts use their student identifiers as demonstration passwords. These credentials are for local demonstration only and must be changed for real use.

Module Description

The ML service performs lazy artifact loading and returns class, pass/fail probabilities, model version, confidence label and top features. For linear models, absolute coefficients are normalized to support feature bars; tree models use their native feature_importances_. Demo endpoints clamp submitted values to feature bounds and return predictions with recommendations. Admin and student login links are deliberately one-click in Demo Mode. CSRF tokens are included in forms and AJAX requests. Live demo UI and predictor are available at `/demo/`; the expected default demo administrator is `admin` / `Admin@1234`.

Testing Methods

Unit tests cover validators, synthetic data generation, model training, probability behavior and service outputs. Integration tests create an isolated temporary SQLite database and exercise demo rendering and seeding, live JSON prediction, administrator authentication, core administrative pages, API health/statistics, chatbot response and student portal access. The completed test run passed 39 tests. Warnings include legacy SQLAlchemy Query.get usage and upstream sklearn/pytest deprecations; these did not fail the suite. Manual smoke testing confirmed the demo page responds over the running Flask server at port 5000 and the model artifacts load. Security-sensitive deployment settings, real institutional SSO and production traffic performance are outside this local test scope.

Chapter 7: Results and Conclusion

Output Screenshots

Figure 7.1 is a high-resolution illustrative rendering of the live interactive demo layout. It documents the visible input and inference panels and should not be interpreted as a browser pixel capture; it is generated for print clarity from the verified application structure and live model evaluation context. The working application at `/demo/` supplies dynamic output from actual model inference. Figure 7.2 presents the measured comparison of the four classifiers.

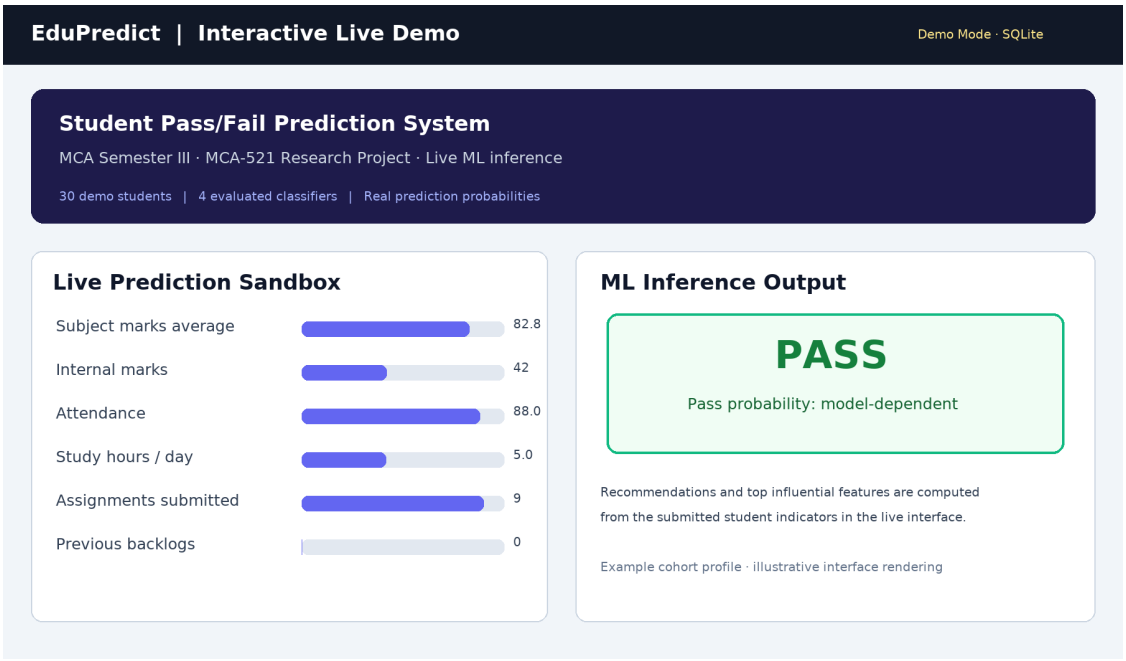


Figure 7.1: High-resolution illustrative rendering of the interactive demo interface.

Results and Observations

The training corpus contains 2,000 records: 67.15% PASS and 32.85% FAIL. The test portion contains 400 records, with support of 269 PASS and 131 FAIL. Logistic Regression records 83.50% accuracy, 86.12% precision, 89.96% recall, 88.00% F1 and 0.9082 ROC-AUC; its five-fold CV accuracy is 84.62% ± 1.80%. Its held-out confusion matrix is TP=242, TN=92, FP=39, FN=27. RBF SVM has 84.75% accuracy and 0.8978 AUC; Gradient Boosting has 83.50% and 0.8967; Random Forest has 83.00% and 0.8866. The system selects Logistic Regression by ROC-AUC, although SVM's observed accuracy is higher. For the FAIL class, Logistic Regression recall is 70.23%, so some at-risk students are missed; this is a material limitation and supports human review rather than automatic decisions. Estimates may shift under new institutions, cohorts or assessment policies.

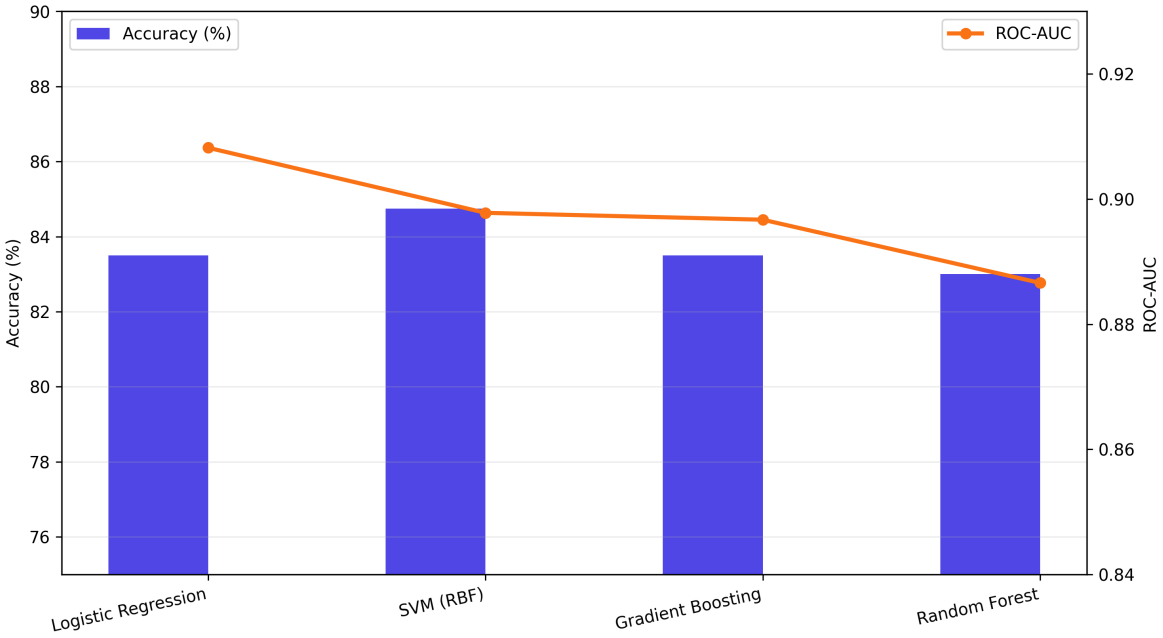


Figure 7.2: Held-out accuracy and ROC-AUC comparison across four classifiers.

Classifier	Accuracy	Precision	Recall	F1	ROC-AUC	CV
Logistic Reg.	83.50%	86.12%	89.96%	88.00%	0.9082	84.62±1.80%
SVM RBF	84.75%	86.62%	91.45%	88.97%	0.8978	82.94±2.13%
Gradient Boosting	83.50%	86.38%	89.59%	87.96%	0.8967	83.44±1.25%
Random Forest	83.00%	84.30%	91.82%	87.90%	0.8866	83.56±0.92%

Table 7.1: Reported classifier evaluation metrics for the supplied dataset and split.

Conclusion

EduPredict demonstrates a complete path from academic data to a persisted, interpretable prediction in a working web application. The implementation compares multiple models, reports threshold-independent discrimination alongside class metrics, and gives administrators and students distinct views. Its SQLite demo allows evaluation without MySQL infrastructure. Results show useful predictive signal in the supplied dataset, but the evaluation is internal to one dataset and one split. Before any real educational use, the model should be externally validated, recalibrated, monitored for group-level error differences, and governed by consent, access control and an explicit human decision policy.

Chapter 8: Future Scope and References

Future Enhancements

Future work can add temporal attendance and assessment trends, institution-specific retraining, probability calibration and threshold tuning to improve early-risk sensitivity. A longitudinal evaluation across semesters should measure stability and intervention outcomes. Further enhancements include role-based access control, password reset and stronger password policy, audit exports, institutional single sign-on, accessible multilingual interfaces, secure deployment, database migrations, privacy retention controls and encrypted backups. Explainability can be extended with validated local explanations and subgroup performance reports. Any intervention workflow should be evaluated with educators and students before adoption.

References

Books: 1. Han, J., Kamber, M. and Pei, J. (2012). Data Mining: Concepts and Techniques, 3rd ed., Morgan Kaufmann. 2. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed., O'Reilly Media.

Research papers: 3. Romero, C. and Ventura, S. (2010). Educational data mining: a review of the state of the art. IEEE Transactions on Systems, Man, and Cybernetics, Part C, 40(6), 601–618. doi:10.1109/TSMCC.2010.2053532. 4. Baker, R.S.J.d. and Inventado, P.S. (2014). Educational data mining and learning analytics. In Learning Analytics, Springer, 61–75. doi:10.1007/978-1-4614-3305-7_4. 5. Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32. doi:10.1023/A:1010933404324.

Websites and technical documentation: 6. Scikit-learn Developers, Model evaluation and scoring, https://scikit-learn.org/stable/modules/model_evaluation.html (accessed 12 Nov 2026). 7. Scikit-learn Developers, StandardScaler, https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html (accessed 12 Nov 2026). 8. Flask documentation, https://flask.palletsprojects.com/ (accessed 12 Nov 2026). 9. Flask-SQLAlchemy documentation, https://flask-sqlalchemy.palletsprojects.com/ (accessed 12 Nov 2026). 10. Python Software Foundation, Python 3 documentation, https://docs.python.org/3/ (accessed 12 Nov 2026). 11. EduPredict project repository, https://github.com/Vaibhav41-cmd/edupredict (accessed 12 Nov 2026).